Proyecto final

UrbanMove: sistema distribuido de movilidad urbana

Computación en la Nube

Grado en Ingeniería Informática
Universidad de Las Palmas de Gran Canaria

1. Introducción

El proyecto final de la asignatura consiste en diseñar, implementar y desplegar un sistema distribuido para una empresa de movilidad urbana que gestiona bicicletas y patinetes eléctricos repartidos por la ciudad. La empresa ofrece una aplicación para usuarios finales y otra aplicación interna para el personal de mantenimiento y operaciones.

El objetivo no es construir una aplicación comercial completa, sino una arquitectura académica suficientemente rica para practicar microservicios atómicos, funciones Lambda, comunicación asíncrona, almacenamiento operacional, Data Lake, data marts, analítica de demanda, tolerancia a fallos, escalabilidad, observabilidad, documentación arquitectónica y automatización DevOps.

El trabajo se realizará en grupos de entre dos y tres personas. Todos los miembros deben participar en código, arquitectura, despliegue, pruebas, documentación y defensa. El reparto de tareas debe quedar documentado en el informe final.

2. Contexto funcional

La empresa dispone de bicicletas y patinetes distribuidos por zonas de la ciudad. Los usuarios pueden registrarse, consultar recursos cercanos, reservar uno, iniciar un viaje, finalizarlo y dejar el recurso en una zona permitida. El personal de mantenimiento puede retirar recursos para recarga, reparación o revisión, y volver a ponerlos a disposición de los usuarios.

Además, el sistema debe recoger eventos de uso y operación en un Data Lake. A partir de esos eventos se construirán data marts orientados a diferentes dominios o microservicios, por ejemplo uso, flota, mantenimiento, reservas, facturación simulada y demanda por zona. Esos data marts deben servir para analizar patrones de uso y generar órdenes de recolocación.

3. Aplicaciones cliente

El sistema debe contemplar al menos tres interfaces diferenciadas: una aplicación para usuarios finales, una aplicación para mantenimiento y operaciones, y un dashboard operativo. No es obligatorio que tengan una interfaz gráfica sofisticada; pueden ser apps web sencillas, APIs documentadas, colecciones de peticiones o clientes CLI. Lo importante es que las responsabilidades estén separadas y que el dashboard muestre información agregada real del sistema.

3.1. Aplicación de usuarios finales

La aplicación de usuarios finales debe permitir:

- Registrar o consultar un usuario.
- Ver bicicletas o patinetes disponibles por zona.
- Crear, confirmar o cancelar una reserva.
- Iniciar el uso de un recurso reservado o disponible.
- Finalizar el viaje indicando zona de destino.
- Consultar el historial personal de reservas y viajes.

3.2. Aplicación de mantenimiento y operaciones

La aplicación de mantenimiento debe permitir:

- Consultar recursos con batería baja, avería o uso anómalo.
- Retirar un recurso de disponibilidad pública.
- Registrar tareas de recarga, reparación o revisión.
- Volver a poner un recurso en servicio.
- Registrar la compra o incorporación de nuevos recursos.
- Consultar órdenes de recolocación generadas por el sistema.
- Marcar una orden de recolocación como asignada, en curso o completada.

3.3. Dashboard operativo

El dashboard debe permitir consultar el estado global del sistema y apoyar decisiones operacionales. Debe alimentarse de eventos, data marts, métricas de los microservicios o una combinación justificada de estas fuentes.

Como mínimo debe mostrar:

- Recursos disponibles, reservados, en uso, en carga, en reparación y retirados.
- Reservas y viajes por franja horaria, zona y tipo de recurso.
- Zonas con déficit o exceso de bicicletas y patinetes.
- Órdenes de mantenimiento y recolocación abiertas, en curso y completadas.
- Indicadores de salud del sistema: servicios activos, errores recientes o eventos fallidos.

4. Arquitectura esperada

La solución debe estar formada por microservicios independientes, funciones Lambda y una capa analítica basada en eventos. La cantidad exacta de componentes puede variar, pero la separación de responsabilidades debe ser clara. Se espera una arquitectura de servicios pequeños, con bajo acoplamiento y una responsabilidad principal por servicio.

La arquitectura debe ser **tolerante a fallos y escalable**. Esto significa que el fallo temporal de un componente no debe romper todo el sistema y que los componentes con más carga deben poder crecer de forma independiente.

4.1. Microservicios atómicos esperados

Microservicio	Responsabilidad
Identidad	Gestiona autenticación, sesiones, tokens y roles básicos. Debe separar usuarios finales, mantenimiento y administración.
Perfiles de usuario	Gestiona datos de usuario, preferencias, validación básica y baja lógica. No debe emitir tokens ni controlar sesiones.
Catálogo de zonas	Mantiene zonas permitidas, restricciones de aparcamiento, áreas de operación y metadatos geográficos simplificados.
Inventario de recursos	Registra bicicletas y patinetes como activos: identificador, tipo, fecha de compra, modelo y estado administrativo.
Disponibilidad	Mantiene qué recursos pueden reservarse en cada momento, evitando mezclar inventario, reservas y mantenimiento.
Batería y telemetría	Recibe o simula nivel de batería, última ubicación, kilómetros o número de usos desde la última revisión.
Reservas	Crea, confirma, cancela y expira reservas. Controla que un recurso no se reserve dos veces.
Viajes	Gestiona inicio y fin de uso. Calcula duración, zona de origen, zona de destino y coste preliminar.
Incidencias	Registra averías, vandalismo, bloqueos, recursos desaparecidos o incidencias reportadas por usuarios y mantenimiento.
Órdenes de mantenimiento	Gestiona retirada, recarga, reparación, revisión, asignación y reincorporación de recursos.
Órdenes de recolocación	Gestiona redistribución de recursos entre zonas a partir de reglas o indicadores de demanda.
Adquisiciones	Registra compra o incorporación de nuevos recursos y pública eventos para inventario y analítica.
Historial	Servicio independiente para consultar histórico de reservas, viajes, incidencias, mantenimiento y operaciones relevantes. No debe ser una tabla secundaria dentro de otro microservicio.
Notificaciones	Envía o simula avisos a usuarios, mantenimiento u operaciones ante reservas, incidencias, órdenes o errores recuperables.
Analítica operacional	Expone consultas agregadas de demanda, ocupación, disponibilidad y rendimiento operacional a partir de data marts. Debe alimentar el dashboard o justificar qué indicadores se calculan en otro componente.

El grupo debe implementar al menos **diez microservicios**. Se valorará que cada microservicio sea atómico: una API clara, una base de datos o almacenamiento propio cuando tenga estado, despliegue independiente y contratos explícitos. Se permite fusionar dos servicios de bajo riesgo si se justifica en el PDF, pero no se pueden fusionar reservas, viajes, historial, mantenimiento ni analítica.

4.2. Documentación arquitectónica obligatoria

La arquitectura no debe quedar solo explicada de palabra. El PDF final debe documentar, como mínimo:

- Vista de contexto: actores, aplicaciones cliente, sistemas externos simulados y límites del sistema.
- Vista de contenedores o despliegue: microservicios, Lambdas, bases de datos, bus de eventos, Data Lake y data marts.
- Vista de componentes para al menos dos microservicios críticos, por ejemplo reservas y viajes.
- Diagramas de secuencia para reserva, inicio de viaje, retirada por mantenimiento y recolocación.
- Tabla de propiedad de datos: qué servicio es dueño de cada entidad o tabla principal.
- Contratos de APIs REST o equivalentes mediante OpenAPI, README estructurado o colección de peticiones.
- Contratos de eventos mediante tabla, JSON Schema, AsyncAPI o documentación equivalente.
- Dos decisiones arquitectónicas documentadas como ADR breve: problema, alternativas, decisión y consecuencias.

La defensa podrá pedir que cualquier diagrama se relacione con código real del repositorio, pipeline DevOps y evidencias de ejecución.

4.3. Funciones Lambda obligatorias

El sistema debe incluir al menos **tres funciones Lambda** o funciones serverless equivalentes. Deben estar conectadas a eventos reales del sistema.

- **Expiración de reservas:** libera un recurso si una reserva no se confirma en el tiempo establecido.
- **Detección de mantenimiento:** genera una orden cuando un recurso queda con batería baja, avería o demasiados usos desde la última revisión.
- **Ingesta analítica:** copia o transforma eventos hacia el Data Lake y actualiza particiones o metadatos.
- **Construcción de data marts:** actualiza tablas agregadas por zona, hora, tipo de recurso o microservicio.
- **Recolocación:** propone órdenes para mover recursos desde zonas con exceso a zonas con déficit.

Tres de estas funciones son obligatorias; el resto se consideran ampliaciones valorables.

5. Estados principales

Cada recurso de movilidad debe tener un ciclo de vida explícito. Como mínimo deben contemplarse los siguientes estados:

- `AVAILABLE`: disponible para reserva o uso.
- `RESERVED`: reservado temporalmente por un usuario.
- `IN_USE`: en uso durante un viaje.
- `LOW_BATTERY`: disponible solo para mantenimiento o pendiente de retirada.
- `WITHDRAWN`: retirado de la vía pública.
- `CHARGING`: en proceso de recarga.
- `REPAIRING`: en reparación.
- `READY_FOR_SERVICE`: listo para volver a la calle.
- `RETIRED`: dado de baja definitivamente.

El grupo debe documentar qué eventos provocan cada transición de estado y qué microservicio es responsable de validarla.

6. Eventos del sistema

La arquitectura debe emitir eventos de dominio. Estos eventos alimentarán procesos asíncronos, Lambdas, historial, Data Lake y data marts.

Evento	**Uso esperado**
UserRegistered	Alta de usuario y creación de perfil.
VehiclePurchased	Incorporación de una bicicleta o patinete nuevo.
VehicleReserved	Reserva creada sobre un recurso disponible.
ReservationCancelled	Reserva cancelada por el usuario.
ReservationExpired	Reserva liberada automáticamente por timeout.
RideStarted	Inicio de uso de un recurso.
RideFinished	Finalización del viaje y liberación o cambio de estado del recurso.
VehicleBatteryLow	Detección de batería baja.
VehicleWithdrawnForMaintenance	Recurso retirado por mantenimiento.
VehicleReturnedToService	Recurso vuelve a estar disponible.
DemandHotspotDetected	Zona con demanda superior a la disponibilidad.
RelocationOrderCreated	Orden creada para mover recursos entre zonas.
RelocationOrderCompleted	Orden completada por el equipo de mantenimiento.

Cada evento debe tener un identificador, timestamp, origen, tipo, versión de esquema y un identificador de correlación que permita seguir un flujo completo.

7. Tolerancia a fallos y escalabilidad

El diseño debe explicar cómo se comporta el sistema cuando hay fallos parciales. No se espera una solución perfecta, pero sí una arquitectura que muestre decisiones conscientes.

7.1. Tolerancia a fallos

El proyecto debe incorporar al menos cuatro de los siguientes mecanismos:

- Reintentos controlados para llamadas o procesamiento de eventos.
- Colas o tópicos para desacoplar microservicios y evitar pérdida de eventos.
- Dead-letter queue o almacenamiento de eventos fallidos para su revisión.
- Operaciones idempotentes para evitar duplicados al reprocesar mensajes.
- Timeouts y gestión explícita de errores entre microservicios.
- Degradación funcional: por ejemplo, permitir finalizar un viaje aunque el servicio de recomendaciones no esté disponible.
- Health checks en microservicios y reinicio automático de contenedores.
- Persistencia de eventos críticos antes de ejecutar procesamiento secundario.

El informe debe incluir al menos dos escenarios de fallo probados o simulados, por ejemplo caída del servicio de historial, error al procesar una orden de mantenimiento o fallo temporal de la Lambda que actualiza data marts.

7.2. Escalabilidad

El proyecto debe justificar cómo escalarían las partes con más carga:

- La app de usuarios y los servicios de reservas y viajes deben poder escalar horizontalmente.
- Las Lambdas deben poder procesar eventos en paralelo.
- El Data Lake debe almacenar eventos particionados para crecer por volumen y tiempo.
- Los data marts deben poder recalcularse de forma incremental o por ventanas temporales.
- La analítica de demanda no debe bloquear las operaciones principales de reserva, inicio o fin de viaje.

El grupo debe identificar al menos dos cuellos de botella probables y proponer una estrategia concreta para mitigarlos.

8. Data Lake y data marts

El proyecto debe incluir una capa analítica separada de los microservicios operacionales. Los microservicios no deben usar el Data Lake como base de datos transaccional. El Data Lake recoge eventos para análisis, auditoría y generación de data marts.

8.1. Data Lake de eventos

El Data Lake puede implementarse con S3, LocalStack, MinIO, ficheros particionados o una alternativa justificada. Debe contener eventos en zonas o capas diferenciadas:

- **Raw:** eventos tal como llegan desde los microservicios o el bus de eventos.
- **Trusted:** eventos validados, normalizados y con esquema conocido.
- **Curated:** datos preparados para consumo analítico y construcción de data marts.

Se recomienda particionar por fecha, tipo de evento y microservicio productor. Por ejemplo:

```
s3://urbanmove-datalake/raw/year=2026/month=09/day=10/source=rides/
s3://urbanmove-datalake/trusted/event_type=RideFinished/
s3://urbanmove-datalake/curated/domain=demand/
```

8.2. Data marts obligatorios

El grupo debe construir al menos **cuatro data marts**. Cada data mart debe tener una finalidad clara y estar asociado a un dominio o microservicio.

Data mart	**Preguntas que debe responder**
Usuarios	£Cuántos usuarios activos hay por período? £Qué usuarios tienen más viajes o cancelaciones?
Reservas	£Cuántas reservas caducan? £En qué zonas se reservan recursos que luego no se usan?
Viajes	£Cuáles son los orígenes y destinos más frecuentes? £Qué duración media tienen los viajes?
Flota	£Cuántos recursos están disponibles, en uso, en carga o en reparación por zona?
Mantenimiento	£Qué recursos requieren recarga o reparación? £Cuánto tiempo pasan fuera de servicio?
Operaciones	£Qué órdenes de recolocación se crean? £Cuántas se completan y con qué retraso?
Demanda	£Qué zonas tienen déficit o exceso de recursos según franja horaria y tipo de vehículo?

Cuatro son obligatorios; se valorará positivamente incluir más si se integran con decisiones reales del sistema.

8.3. Uso analítico mínimo

La analítica debe alimentar al menos una decisión operacional. Como mínimo, el sistema debe generar una orden de recolocación a partir de indicadores de demanda.

Una regla sencilla podría ser:

> Si una zona tiene menos de tres recursos disponibles y ha tenido más de diez intentos de reserva o viajes iniciados en la última hora, se genera una orden para recolocar recursos desde una zona con exceso.

La regla puede ser más avanzada si el grupo incorpora predicción, clustering de zonas o análisis temporal.

9. Despliegue

El proyecto puede desplegarse en AWS Academy, AWS real, LocalStack o un entorno local con servicios equivalentes. La decisión debe justificarse en el PDF final.

Se valorará especialmente:

- Contenedores desplegados en ECS, App Runner, Kubernetes, Docker Compose o alternativa justificada.
- Lambdas desplegadas con SAM, CDK, Terraform, Serverless Framework o scripts reproducibles.
- Uso de SQS, SNS, EventBridge, Kafka, RabbitMQ o equivalente para eventos.
- Data Lake sobre S3, LocalStack, MinIO o almacenamiento particionado reproducible.
- Infraestructura definida con Terraform, CloudFormation, SAM, CDK o scripts versionados.
- Configuración de escalado horizontal, concurrencia o réplicas cuando la tecnología elegida lo permita.
- Health checks, reinicios automáticos o alarmas para detectar fallos.

10. DevOps y observabilidad

El repositorio debe incluir una práctica DevOps visible y reproducible. No basta con tener código que funcione manualmente: el grupo debe demostrar cómo se construye, prueba y despliega el sistema.

Como mínimo, el pipeline debe:

- Ejecutar pruebas automáticas de varios microservicios en cada cambio.
- Construir imágenes Docker para los microservicios desplegables o, al menos, para los servicios críticos.
- Validar formato, estilo o análisis estático de código.
- Empaquetar o desplegar Lambdas mediante comandos versionados.
- Ejecutar migraciones, semillas de datos o preparación de infraestructura de forma documentada.
- Publicar artefactos, logs de pipeline, capturas o evidencias de ejecución.
- Evitar imprimir credenciales o secretos en logs.

Se recomienda organizar el repositorio como monorepo con carpetas independientes por servicio, por ejemplo services/reservations, services/rides, services/history, lambdas/, infra/ y docs/. Si el grupo usa varios repositorios, debe justificarlo y documentar cómo se versionan conjuntamente.

El README debe permitir levantar una versión local con un único flujo razonable de comandos, por ejemplo docker compose up, scripts de inicialización y comandos de prueba.

La observabilidad debe permitir diagnosticar un flujo completo desde la app de usuarios o mantenimiento hasta los eventos y data marts. Como mínimo debe existir:

- Logs estructurados con identificador de correlación.
- Métricas de reservas, viajes, recursos disponibles y órdenes de mantenimiento.
- Evidencia de eventos enviados y consumidos.
- Consulta o captura de algún data mart generado.
- Dashboard operativo con indicadores funcionales y técnicos actualizados durante la demostración.

11. Flujos obligatorios para la demostración

La defensa debe mostrar, como mínimo, cinco flujos completos.

11.1. Flujo de usuario

1. El usuario se registra o inicia sesión.
2. Consulta bicicletas o patinetes disponibles en una zona.
3. Reserva un recurso.
4. Inicia el viaje.
5. Finaliza el viaje en otra zona.
6. Se genera historial y eventos para el Data Lake.

11.2. Flujo de mantenimiento

1. El sistema detecta un recurso con batería baja o avería.
2. Se genera una orden de mantenimiento.
3. La app de mantenimiento retira el recurso.
4. Se registra recarga o reparación.
5. El recurso vuelve a estar disponible.
6. El historial y los data marts reflejan el cambio.

11.3. Flujo de analítica y recolocación

1. Se simulan viajes o reservas en varias zonas.
2. Los eventos se almacenan en el Data Lake.
3. Se actualiza al menos un data mart de demanda.
4. El sistema detecta déficit de recursos en una zona.
5. Se crea una orden de recolocación.
6. La app de mantenimiento consulta y completa la orden.

11.4. Flujo de dashboard

1. Se generan datos de uso, mantenimiento y recolocación.
2. Se actualiza al menos un data mart o vista agregada.
3. El dashboard muestra indicadores antes y después de la actualización.
4. El grupo explica qué servicios, eventos y fuentes de datos alimentan cada indicador.

11.5. Flujo de fallo controlado

1. Se provoca o simula la indisponibilidad temporal de un componente no crítico.
2. El sistema registra el error con identificador de correlación.
3. Los eventos críticos no se pierden.
4. El sistema se recupera o permite reprocesar el evento.
5. El grupo explica qué parte de la arquitectura evita que el fallo se propague.

12. Trabajo por grupo

El reparto de trabajo debe estar equilibrado entre los integrantes. Todos deben participar en la implementación, documentación y defensa. Una distribución orientativa sería:

- Persona A: app de usuarios, identidad, perfiles, reservas, viajes y pruebas asociadas.
- Persona B: app de mantenimiento, inventario, disponibilidad, incidencias, mantenimiento, recolocación y Lambdas asociadas.
- Persona C, si el grupo tiene tres integrantes: historial, Data Lake, data marts, analítica operacional y dashboard. En grupos de dos, estas tareas se reparten entre A y B.
- Trabajo conjunto: arquitectura, contratos, eventos, CI/CD, despliegue, pruebas, observabilidad, informe y defensa.

13. Entregables

La entrega debe incluir:

1. Repositorio GitHub con todo el código fuente.
2. Documento PDF final explicando el trabajo realizado, arquitectura, decisiones técnicas, reparto, pruebas, evidencias, problemas encontrados y limpieza de recursos.
3. Documentación arquitectónica completa: contexto, contenedores o despliegue, componentes de servicios críticos, secuencias, propiedad de datos y ADRs.
4. Diagrama de arquitectura que muestre apps cliente, dashboard, microservicios atómicos, Lambdas, bus de eventos, Data Lake y data marts.
5. Documentación de APIs: endpoints, métodos, payloads y códigos de respuesta principales.
6. Documentación de eventos: nombre, productor, consumidores y esquema mínimo.
7. Descripción de los data marts construidos y ejemplos de consultas o indicadores.
8. Dashboard operativo con capturas o evidencia de ejecución explicadas.
9. Instrucciones de ejecución local.
10. Instrucciones de despliegue o reproducción de infraestructura.
11. Evidencia de CI/CD funcionando: ejecución de pruebas, construcción de imágenes o artefactos, y validaciones automáticas.
12. Evidencia de observabilidad: logs, métricas, trazas, paneles o capturas explicadas.

14. Mínimos para aprobar

Para superar el proyecto deben cumplirse todos los requisitos siguientes:

- Tres interfaces diferenciadas: usuarios finales, mantenimiento/operaciones y dashboard operativo.
- Al menos diez microservicios atómicos, incluyendo obligatoriamente identidad, perfiles de usuario, inventario, disponibilidad, reservas, viajes, historial, órdenes de mantenimiento, órdenes de recolocación y analítica operacional.
- El historial debe estar implementado como microservicio independiente.
- Documentación arquitectónica con diagramas, contratos de APIs, contratos de eventos, propiedad de datos y al menos dos ADRs.
- Al menos tres Lambdas o funciones serverless equivalentes.
- Al menos un bus, cola o tópico para comunicación asíncrona.
- Data Lake de eventos con separación raw, trusted o curated, aunque sea simplificada.
- Al menos cuatro data marts asociados a dominios o microservicios.

- Generación de órdenes de recolocación a partir de datos de demanda.
- Dashboard con indicadores de flota, reservas, viajes, mantenimiento, recolocación y salud básica del sistema.
- Mecanismos explícitos de tolerancia a fallos y al menos dos fallos simulados o probados.
- Justificación de escalabilidad horizontal o por eventos en los componentes de mayor carga.
- Pipeline DevOps con pruebas automáticas, construcción de artefactos o imágenes, y evidencias de ejecución.
- PDF final y defensa técnica por parte de todos los miembros.

15. Ampliaciones valorables

Se valorarán positivamente:

- Autenticación y roles separados para usuarios y mantenimiento.
- Uso de Terraform para recursos cloud.
- Despliegue de contenedores en ECS y publicación de imágenes en ECR.
- Uso de S3, DynamoDB, SQS, SNS, EventBridge, Lambda y CloudWatch.
- Contratos OpenAPI y AsyncAPI versionados para microservicios y eventos.
- Pruebas de carga con Locust sobre reservas o viajes.
- Dashboard avanzado con filtros por zona, ventana temporal, predicción de demanda o comparación antes/después de recolocaciones.
- Modelo predictivo sencillo para anticipar demanda por zona.
- Manejo de mensajes fallidos, reintentos o dead-letter queue.
- Despliegue por entornos, blue/green simulado o rollback documentado.

16. Rúbrica

Criterio	**Peso**	**Descripción**
Arquitectura	16 %	Separación clara entre apps, microservicios atómicos, Lambdas, eventos, Data Lake y data marts. Coherencia de responsabilidades, datos propios y contratos.
Implementación	16 %	Servicios funcionales, APIs consistentes, gestión correcta de estados, persistencia y empaquetado reproducible.
Eventos y serverless	12 %	Eventos de dominio bien definidos, consumidores asíncronos, Lambdas integradas y trazabilidad de flujos.
Capa analítica	14 %	Data Lake de eventos, data marts útiles, indicadores de demanda, dashboard operativo y generación razonada de órdenes de recolocación.
Tolerancia a fallos	10 %	Reintentos, colas, idempotencia, DLQ, degradación funcional y escenarios de fallo probados o simulados.

Escalabilidad	9 %	Escalado horizontal, procesamiento paralelo, particionado de datos y separación entre operación y analítica.
DevOps y despliegue	10 %	CI/CD, pruebas automáticas, construcción de imágenes o artefactos, despliegue reproducible e infraestructura versionada.
Observabilidad	6 %	Logs, métricas, dashboard, evidencias de eventos y capacidad de diagnóstico extremo a extremo.
Documentación y defensa	7 %	PDF final, diagramas C4 o equivalentes, contratos, ADRs, reparto equilibrado y defensa técnica de todos los miembros.

17. Seguridad y limpieza de recursos

No deben incluirse credenciales, tokens, claves privadas ni identificadores sensibles en el repositorio o en el PDF. Si se usan servicios cloud reales, el informe debe indicar qué recursos se han creado, cuáles se han eliminado y si queda alguno activo.

El PDF final debe incluir una sección específica con:

- Recursos creados.
- Recursos eliminados.
- Recursos que quedan activos, si los hubiera.
- Coste esperado o motivo por el que no existe coste.
- Medidas adoptadas para no exponer credenciales.

18. Defensa

La defensa tendrá una duración orientativa de 10 minutos por grupo. Debe incluir una demostración funcional de los cinco flujos obligatorios, una explicación de la arquitectura y preguntas individuales a todos los miembros.

Durante la defensa se podrá que se explique:

- Por qué cada microservicio existe como componente independiente.
- Cómo se evita reservar dos veces el mismo recurso.
- Qué ocurre cuando una reserva caduca.
- Cómo llega un evento al Data Lake.
- Qué data mart se usa para decidir una recolocación.
- Cómo se diagnostica un fallo entre microservicios.
- Qué componentes pueden escalar de forma independiente y por qué.
- Qué ocurre si se duplica un evento o si una Lambda falla temporalmente.